

Lenguajes de Marcas y Sistemas de Gestión de Información

CSS

Introducción

**1º ADMINISTRACIÓN DE
SISTEMAS INFORMÁTICOS
EN RED (A.S.I.R.)
2023-2024**

1 - Que son las hojas de estilo (CSS)?

CSS son las siglas de **Cascade Style Sheet** que traducido significa hojas de estilo en cascada.

Las hojas de estilo son una tecnología que nos permite controlar la apariencia de una página web. En un principio, los sitios web se concentraban más en su contenido que en su presentación.

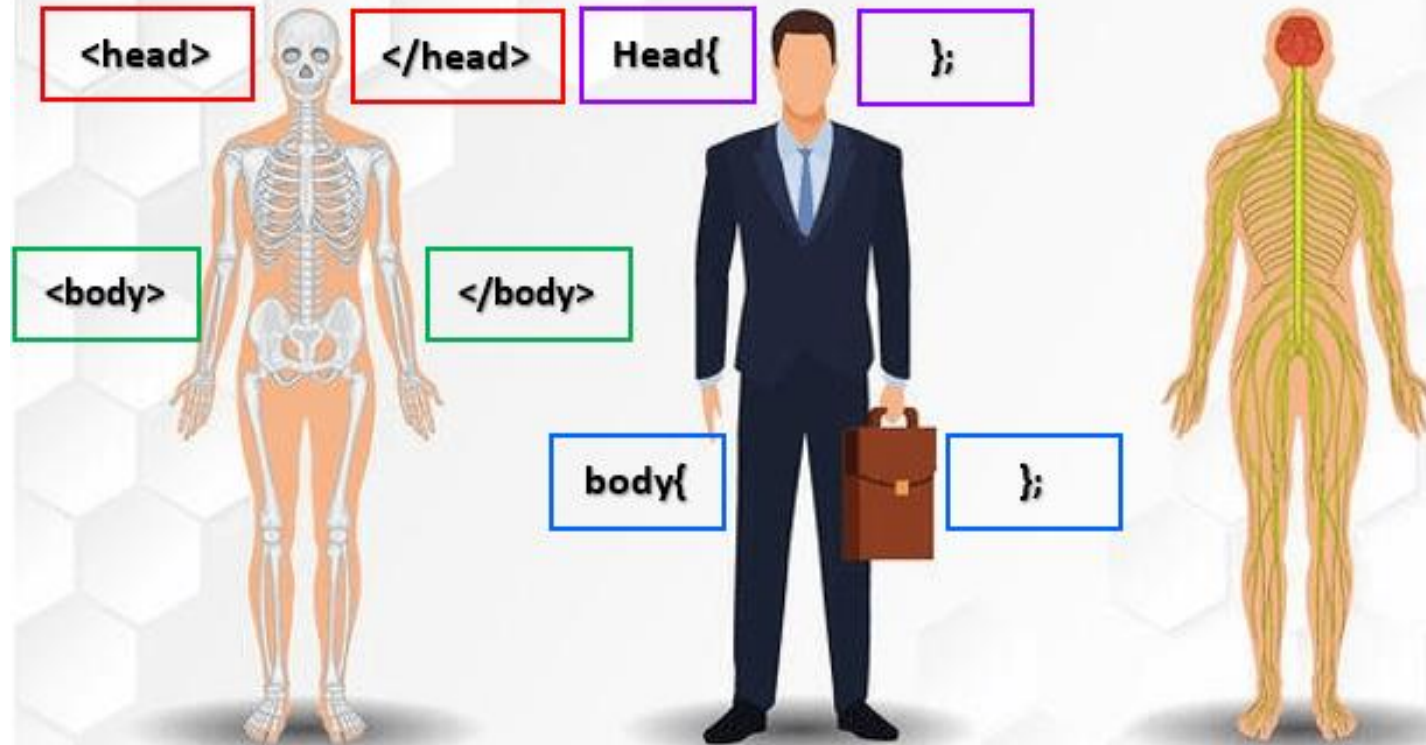
HTML no pone atención en la apariencia del documento, sino en la estructura. CSS describe como los elementos dispuestos en la página son presentados al usuario. CSS es un gran avance que complementa el HTML y la Web en general.

Con **CSS podemos especificar estilos como el tamaño, fuentes, color, espaciado entre textos y recuadros así como el lugar donde disponer texto e imágenes en la página.**

HTML

CSS

JAVASCRIPT



El lenguaje de las Hojas de Estilo está definido en la Especificaciones CSS1, CSS2 y CSS3 del World Wide Web Consortium (W3C), es un estándar aceptado por toda la industria relacionada con la Web, o por lo menos, gran parte de navegadores. Podemos visitar W3C.

Veremos que **podemos asociar las reglas de estilo a las marcas HTML de tres maneras: directamente a la marca, en el head de la página o agrupar las reglas de estilo en un archivo independiente con extensión *.css**

En este curso veremos las tres metodologías, pero pondremos énfasis en la tercera forma, que es la más adecuada para separar el contenido de la página y la forma como se debe representar la misma por medio de la hoja de estilo.

El curso brinda un concepto teórico corto, luego un problema resuelto que debemos ejecutar, modificar y jugar con el mismo. Por último, y lo más importante, un ejercicio propuesto que nos permitirá saber si podemos aplicar el concepto.

Problema: El objetivo de esta pequeña página es ver como afecta el estilo definido a la marca p (párrafo)

```
<!DOCTYPE html>
<html>
<head>
  <title>Prueba de hojas de estilo</title>
  <meta charset="UTF-8">
</head>
<body>
<p style="color:#000000;background-color:yellow;font-family:verdana;font-size:25px;text-align:center">Esto es un ejemplo</p>
</body>
</html>
```

ijes%20de%20marcas%20y%20sistemas%20de%20gestion%

Esto es un ejemplo

2 - Definición de estilos a nivel de elemento HTML.

Es la forma más fácil pero **menos recomendada** para aplicación de un estilo a un elemento HTML. Se define en la propiedad style los estilos a definir para dicho elemento.

Es común a veces definir estilos directamente en los elementos HTML cuando estamos probando diseños de elementos particulares de la página y posteriormente trasladar el estilo creado a la zona de definición de estilos.

La sintaxis para definir un estilo a un elemento HTML es la siguiente:

```
<!DOCTYPE html>
<html>
<head>
  <title>Título de la página</title>
  <meta charset="UTF-8">
</head>
<body>
<h1 style="color:#ff0000;background-color:#ffff00">
  Este mensaje es de color rojo sobre fondo amarillo.
</h1>
</body>
</html>
```

El resultado en el navegador al cargar esta página es:



Este mensaje es de color rojo sobre fondo amarillo.

Por defecto, **todo navegador tiene un estilo definido para cada elemento HTML**, lo que hacemos con la propiedad style es redefinir el estilo por defecto. En este problema definimos que el elemento h1 defina como color de texto el rojo y como color de fondo el amarillo:

```
<h1 style="color:#ff0000;background-color:#ffff00">
```

Este mensaje es de color rojo sobre fondo amarillo.

```
</h1>
```

Veremos más adelante que hay muchas propiedades en CSS. En este primer ejemplo inicializamos las propiedades color (define el color del texto) y background-color (define el color de fondo del texto).

Cada vez que **inicializamos una propiedad debemos separarla** de la siguiente **por punto y coma**.

Cuando definimos estilos directamente en el elemento HTML, tenemos que tener en cuenta que **el estilo se aplica únicamente a dicho elemento** donde inicializamos la propiedad style, es decir, **si tenemos dos secciones h1, deberemos definir la propiedad style para cada elemento**:

```
<h1 style="color:#ff0000;background-color:#ffff00">
```

Primer título

```
</h1>
```

```
<h1 style="color:#ff0000;background-color:#ffff00">
```

Segundo título

```
</h1>
```

Como podemos observar, más allá que los dos estilos son exactamente iguales, estamos obligados a **definirlos para cada elemento HTML**.

Problema: Crear una página HTML y definir tres títulos empleando los elemento h1, h2 y h3. Definir el mismo color para el texto de cada bloque y colores diferentes para el fondo de los mismos (cada elemento HTML debe tener definida la propiedad style)

3 - Definición de estilos a nivel de página.

También podemos hacer la definición de estilos para los distintos elementos HTML de la página en una **sección especial de la cabecera que la encerramos entre las marcas HTML** (en su interior definimos los estilos para los elementos HTML que necesitamos):

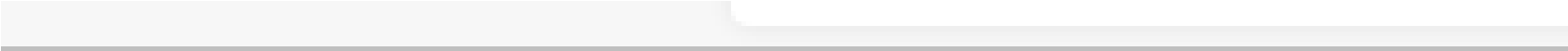
```
<style>  
</style>
```

El problema del concepto anterior donde debíamos crear un estilo similar para el elemento h1 se puede resolver en forma más adecuada empleando la definición de estilos a nivel de página.

Problema: Mostrar dos títulos con texto de color rojo sobre fondo amarillo.

```
<!DOCTYPE html>
<html>
<head>
  <title>Problema</title>
  <meta charset="UTF-8">
<style>
  h1 {color:#ff0000;background-color:#ffff00}
</style>
</head>
<body>
<h1>Primer título</h1>
<h1>Segundo título</h1>
</body>
</html>
```

El resultado en el navegador al cargar esta página es:



Primer título

Segundo título

Podemos observar que en la cabecera de la página definimos la sección:

```
<style>  
h1 {color:#ff0000;background-color:#ffff00}  
</style>
```

Debe estar encerrada por el elemento style. En este ejemplo indicamos al navegador que en todos los lugares de esta página donde se utilice el elemento h1 debe aplicar como estilo de color de texto el rojo y fondo el amarillo. Podemos observar que es mucho más eficiente que definir los estilos directamente sobre los elementos HTML dentro del cuerpo de la página.

Podemos definir estilos para muchos elementos en la sección style:

```
<!DOCTYPE html>
<html>
<head>
  <title>Problema</title>
  <meta charset="UTF-8">
<style>
  h1 {color:#ff0000}
  h2 {color:#00ff00}
  h3 {color:#0000ff}
</style>
</head>
<body>
<h1>rojo</h1>
<h2>verde</h2>
<h3>azul</h3>
</body>
</html>
```

rojo

verde

azul

Problema: Definir un estilo diferente para los elementos h1,h2,h3,h4,h5 y h6. Mostrar mensajes utilizando estos elementos a los que se le han definido estilos (por ahora solo conocemos el color de texto y fondo)

4 - Propiedades relacionadas a fuentes.

Contamos con una serie de propiedades relacionadas a fuentes:

font-family → Establece la fuente o tipo de letra (Verdana, Arial, new roma, etc.)

font-size → Especifica el tamaño de la letra (250%, 16px)

font-style → permite definir el aspecto de una familia tipográfica entre los valores: normal, italic (cursiva) y oblique.

font-weight → Especifica el peso o grueso de la letra. Algunos tipos de letra sólo están disponibles en normal y bold.

font-variant → Establece la variante del tipo de letra. (normal, small-caps)

Podemos inicializar algunas o todas las propiedades relacionadas a fuentes, un ejemplo:

```
<!DOCTYPE html>
<html>
<head>
  <title>Problema</title>
  <meta charset="UTF-8">
<style>
h1 {
  font-family:times new roman;
  font-size:30px;
  font-style:italic;
  font-weight:bold;
}
h2 {
  font-family:verdana;
  font-size:20px;
}
</style>
</head>
<body>
<h1>Titulo de nivel 1</h1>
<h2>Titulo de nivel 2</h2>
</body>
</html>
```

Titulo de nivel 1

Titulo de nivel 2

Como podemos observar, hemos definido dos reglas de estilos para los elementos h1 y h2, eso significa que el navegador utilizará esas reglas de fuentes para todas las partes de la página que se utilicen dichos elementos HTML.

La primera regla definida para el elemento h1 es:

```
h1 {  
    font-family:times new roman;  
    font-size:30px;  
    font-style:italic;  
    font-weight:bold;  
}
```

Recordemos que **para definir la regla de estilo debemos indicar el nombre del elemento HTML a la que definiremos el estilo** (en este caso h1) **y luego, entre llaves, todas las propiedades y sus valores separados por punto y coma.**

La primera propiedad inicializada es font-family, algunas de las fuentes más comunes a las que puede acceder el navegador son:

Arial

Arial Black

Arial Narrow

Courier New

Georgia

Impact

Tahoma

Times New Roman

Verdana

En caso que la fuente no esté disponible el navegador selecciona el estilo por defecto para ese elemento HTML.

Podemos definir varias fuentes por si acaso alguna no se encuentra disponible para el navegador, esto lo hacemos separando por coma todas las fuentes (se eligen de izquierda a derecha):

font-family: verdana, arial, georgia;

La segunda propiedad inicializada es **font-size**, hay varias medidas para indicar el tamaño de la fuente (veremos más adelante otros sistemas de medida), en nuestro caso indicamos **en píxeles**:

```
font-size:30px;
```

La tercera propiedad es font-style, que puede tener los siguientes valores :

Normal, italic, oblique

La última propiedad es font-weight (peso de la fuente), pudiendo tomar los siguientes valores:

Normal, bold, bolder, lighter, 100, 200, 300, 400, 500, 600, 700, 800, 900

Esta propiedad indica el peso de la fuente (**cuanto mayor sea el valor, los caracteres serán más rellenos**)

La segunda regla define sólo dos propiedades relacionadas a la fuente:

```
h2 {  
    font-family:verdana;  
    font-size:20px;  
}
```

Las propiedades que no se inicializan, serán inicializadas por el navegador con el valor por defecto que este tenga.

Existe una última propiedad no utilizada en este ejemplo que es font-variant que puede asumir estos dos valores:

small-caps
Normal

Define si la fuente se muestra en mayúsculas tipo normal o pequeñas.

Problema:

Definir reglas para los elementos HTML: h1,h2,h3,h4,h5 y h6. Inicializar la propiedad font-size con valores decrecientes para cada uno de los elementos (40,30,25,20,15 y 10 píxeles). Inicializar la propiedad font-family para las tres primeros elementos con los valores: Arial, Arial Black y Arial Narrow.

5 - Agrupación de varios elementos HTML con una misma regla de estilo.

Esta característica nos permite ahorrar la escritura de reglas duplicadas para diferentes elementos de HTML.

La sintaxis es disponer los nombres de los elementos HTML que queremos aplicar una regla separados por comas:

```
h1,h2,h3 {  
  font-family:verdana;  
  color:#0000ff;  
}
```

Supongamos que queremos la misma fuente y color para los elementos h1,h2 y h3 luego podemos implementarlo de la siguiente manera:

```
<!DOCTYPE html>
<html>
<head>
  <title>Problema</title>
  <meta charset="UTF-8">
```

```
<style>
h1,h2,h3 {
  font-family:verdana;
  color:#0000ff;
}
</style>
</head>
<body>
<h1>Título de nivel 1</h1>
<h2>Título de nivel 2</h2>
<h3>Título de nivel 3</h3>
</body>
</html>
```

Título de nivel 1

Título de nivel 2

Título de nivel 3

Es decir, **separamos por coma todas los elementos a los que se aplicará la misma regla de estilo:**

```
h1,h2,h3 {  
  font-family:verdana;  
  color:#0000ff;  
}
```

Problema: Definir la misma fuente, color y tamaño de fuente para los elementos p (párrafo) y h6 (tener en cuenta que cuando vea la página el texto que se encuentra en el elemento h6 difiere del texto que se encuentra dentro del párrafo por la propiedad font-weight (ya que el elemento h6 es de tipo bold por defecto y el elemento p tiene por defecto normal))

6 - Definición de varias reglas para un mismo elemento HTML.

En algunas circunstancias, es necesario desglosar la inicialización de propiedades en distintas reglas.

Podemos definir más de una regla para un elemento HTML, en este ejemplo el elemento h1 tiene dos reglas:

```
h1,h2,h3,h4,h5,h6 {  
    font-family:Verdana;  
}  
h1 {  
    font-size:40px;  
}
```

Supongamos que queremos todas las cabeceras con la misma fuente pero tamaños de fuente distinta, luego podemos implementarlo de la siguiente manera:

```
<!DOCTYPE html>
<html>
<head>
  <title>Problema</title>
  <meta charset="UTF-8">
<style>
h1,h2,h3,h4,h5,h6 {
  font-family:Verdana;
}
h1 {
  font-size:40px;
}
h2 {
  font-size:30px;
}
h3 {
  font-size:25px;
}
h4 {
  font-size:20px;
}
h5 {
  font-size:15px;
}
```

```
h6 {  
  font-size:10px;  
}  
</style>  
</head>  
<body>  
<h1>Título de nivel 1</h1>  
<h2>Título de nivel 2</h2>  
<h3>Título de nivel 3</h3>  
<h4>Título de nivel 4</h4>  
<h5>Título de nivel 5</h5>  
<h6>Título de nivel 6</h6>  
</body>  
</html>
```

Título de nivel 1

Título de nivel 2

Título de nivel 3

Título de nivel 4

Título de nivel 5

Título de nivel 6

Es decir, podemos inicializar un conjunto de elementos HTML con una serie de propiedades de estilo comunes. Luego agregamos en forma individual las propiedades no comunes a dichas marcas:

```
h1,h2,h3,h4,h5,h6 {  
    font-family:Verdana;  
}  
h1 {  
    font-size:40px;  
}
```

Es decir, al elemento h1 realmente le hemos definido 2 propiedades: font-family y font-size. Lo mismo para los otros elementos HTML.

Problema:

Definir para el elemento `<h6>` el mismo tamaño de fuente (12px) que el elemento `<p>` (párrafo) pero con color azul para el título y gris claro (color:#aaaaaa) para el párrafo. Agrupar los dos elementos para la definición de la fuente y posteriormente agregar a cada elemento el valor respectivo para el color del texto.

7 - Propiedades relacionadas al texto (color, text-align, text-decoration)

A una de estas propiedades ya la vimos cuando comenzamos con los primeros conceptos: color, nos permite definir el color del texto, lo podemos indicar por medio de **tres valores hexadecimales que indican la mezcla de rojo, verde y azul**. Por ejemplo si queremos **rojo** puro debemos indicar:

```
color:#ff0000;
```

Si queremos **verde** puro:

```
color:#00ff00
```

Si queremos **azul** puro:

```
color:#0000ff
```

Luego si queremos amarillo debemos mezclar el rojo y el verde:

```
color:#ffff00
```

Hay muchos programas que nos permiten seleccionar un color y nos descomponen dicho valor en las proporciones de rojo, verde y azul.

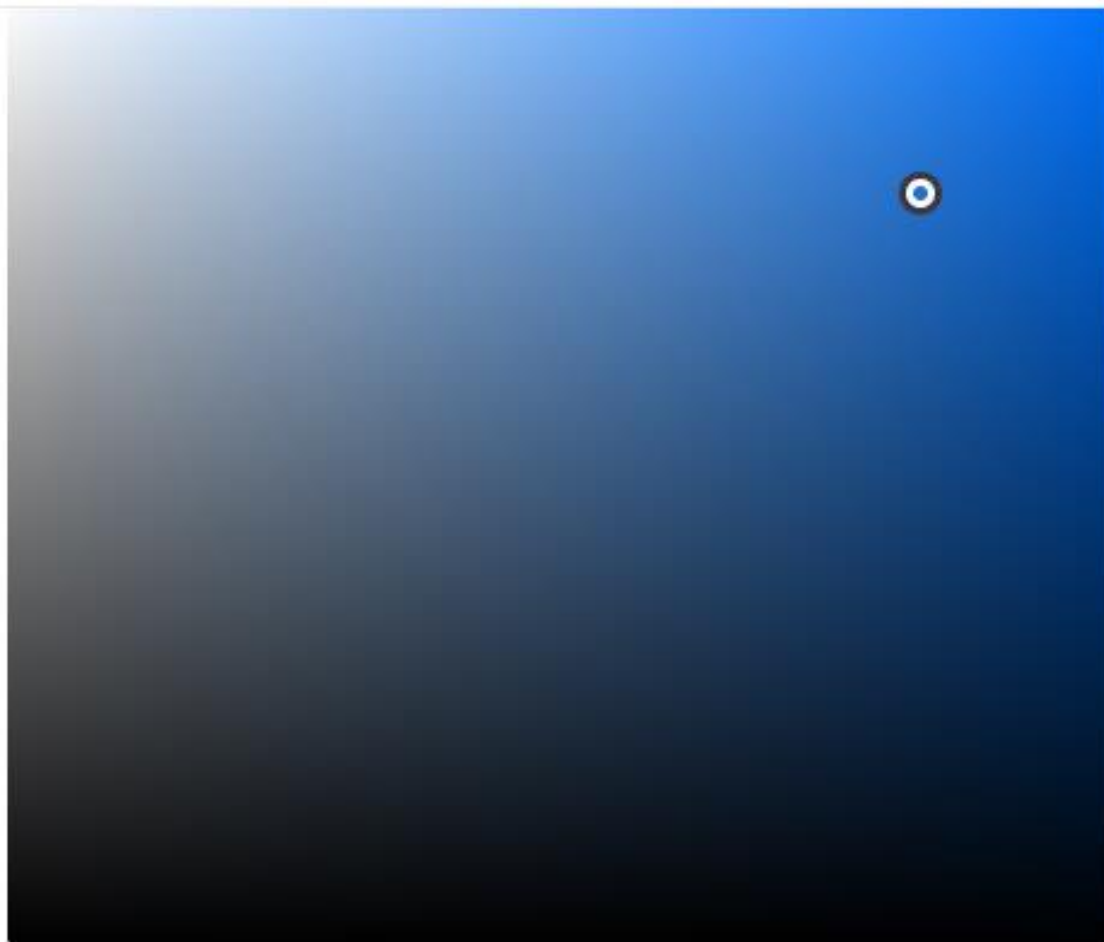
<https://htmlcolorcodes.com/es/>



HEX **#236FCA**

RGB **35, 111, 202**

HSL **213, 83%, 46%**



236FCA

R **35**

G **111**

B **202**

H **213**

S **83**

L **46**

C **83**

M **45**

Y **0**

K **21**

Otra forma de indicar el color, si tenemos los valores en decimal, es por medio de la siguiente sintaxis:

color:rgb(255,0,0);

Es decir, por medio de la función rgb(red,green,blue), indicamos la cantidad de rojo, verde y azul en formato decimal.

La segunda propiedad relacionada al texto es text-align, que puede tomar alguno de estos cuatro valores:

left

right

center

justify

Si especificamos:

text-align:center;

El texto aparecerá centrado. Si queremos justificar a derecha, emplearemos el valor right y si queremos a la izquierda, el valor será left.

La tercera propiedad relacionada con el texto que podemos emplear es **text-decoration** que nos permite entre otras cosas que **aparezca subrayado el texto, tachado o una línea en la parte superior**, los valores posibles de esta propiedad son:

none

underline

overline

line-through

Como ejemplo, definiremos estilos relacionados al texto para los elementos de cabecera h1, h2 y h3:

Título de nivel 1.

Título de nivel 2.

Título de nivel 3.

Es decir, para los títulos de nivel 1 tenemos la regla:

```
h1 {  
  color:#ff0000;  
  text-align:left;  
  text-decoration:underline;  
}
```

Color de texto rojo intenso, el texto debe aparecer de izquierda a derecha y subrayado.

Luego para el elemento h2 tenemos:

```
h2 {  
  color:#dd0000;  
  text-align:center;  
  text-decoration:underline;  
}
```

El color sigue siendo rojo pero un poco más oscuro, el texto debe aparecer centrado y subrayado.

Y por último:

```
h3 {  
  color:#aa0000;  
  text-align:right;  
  text-decoration:underline;  
}
```

Para los títulos de nivel tres, el texto es rojo más oscuro, alineado a derecha y subrayado.

Problema:

Definir para el elemento párrafo (p) el color verde puro, texto centrado. Imprimir varios párrafos para ver los resultados según la regla de estilo definida. ¿Qué valor debemos definir para que el texto aparezca con color rojo oscuro?

8 - Más propiedades relacionadas al texto (letter-spacing, word-spacing, text-indent, text-transform)

Vimos en el concepto anterior las propiedades:

color

text-align

text-decoration

Ahora veremos el objetivo de las siguientes propiedades que nos faltan:

letter-spacing

word-spacing

text-indent

text-transform

La propiedad **letter-spacing** y **word-spacing** permiten indicar el **espacio** que debe haber **entre los caracteres y entre las palabras**.

La propiedad **text-indent**, **indenta la primera línea de un texto**. A partir de la segunda línea, el texto aparece sin indentación. **Podemos indicar un valor negativo con lo que la indentación es hacia la izquierda**.

Por último, la propiedad text-transform puede inicializarse con alguno de los siguientes valores:

none - capitalize - lowercase - uppercase

Cada uno de estos valores transforman el texto como sigue:

capitalize: Dispone en mayúsculas el primer caracter de cada palabra.

lowercase: Convierte a minúsculas todas las letras del texto.

uppercase: Convierte a mayúsculas todas las letras del texto.

none: No provoca cambios en el texto.

El siguiente ejemplo define reglas para los elementos h1 y p:

```
<!DOCTYPE html>
<html>
<head>
  <title>Problema</title>
  <meta charset="UTF-8">
<style>
h1 {
  letter-spacing:10px;
  word-spacing:30px;
  text-transform:capitalize;
}
p {
  text-indent:20px;
}
</style>
</head>
```

```
<body>
```

```
<h1>Este es un título de nivel 1</h1>
```

```
<p>Todo el texto siguiente se encuentra encerrado en el elemento de párrafo  
y con el objetivo de ver el efecto de la propiedad text-indent.
```

```
La propiedad text-indent podemos inicializarla con un valor positivo,  
como es el caso actual o podemos inicializarla con un valor negativo lo que  
provocará que el texto de la primera línea del párrafo comience en una columna  
inferior al de todo el bloque. </p>
```

```
</body>
```

```
</html>
```

Este Es Un Título De Nivel 1

Todo el texto siguiente se encuentra encerrado en el elemento de párrafo y con el objetivo de ver el efecto de la propiedad text-indent. La propiedad text-indent podemos inicializarla con un valor positivo, como es el caso actual o podemos inicializarla con un valor negativo lo que provocará que el texto de la primera línea del párrafo comience en una columna inferior al de todo el bloque.

La cabecera de nivel uno, tiene la siguiente regla:

```
h1 {  
  letter-spacing:10px;  
  word-spacing:30px;  
  text-transform:capitalize;  
}
```

Es decir que las letras deben tener una separación de 10 pixeles, las palabras deben estar separadas por 30 pixeles y por último deben disponerse en mayúsculas la primera letra de cada palabra.

Para el elemento p tenemos la siguiente regla:

```
p {  
  text-indent:20px;  
}
```

Es decir, la primera línea del párrafo deberá imprimirse 20 píxeles a la derecha donde normalmente debería aparecer.

Problema:

Definir los elementos de cabecera h1, h2 y h3 con valores decrecientes para las propiedades:

letter-spacing

word-spacing

Luego inicializar la propiedad text-transform para que el texto siempre salga en mayúsculas.

9 - Herencia de propiedades de estilo.

Ahora veremos que el conjunto de elementos HTML forman en sí un árbol que en su raíz podemos identificar el elemento body del cual se desprenden otros elementos contenidos en esta sección, como podrían ser los elementos h1,h2,h3,h4,h5,h6,p,div que contienen en su interior otros elementos HTML como podrían ser em, strong, pre etc.

Con ejemplos vemos que **hay muchos estilos que se heredan** (todos los vistos hasta el momento se heredan), es decir **si definimos la propiedad color para el elemento h1**, luego **si dicho elemento incorpora un texto con el elemento "em" en su interior, la propiedad color del elemento "em" tendrá el mismo valor que la propiedad h1** (es decir el elemento em hereda las propiedades del elemento h1)

Con un ejemplo veremos el resultado de la herencia de propiedades entre los elementos HTML:

```
<html>
<head>
  <title>Problema</title>
  <meta charset="UTF-8">
<style>
body {
  color:#0000ff;
  font-family:verdana;
}
</style>
</head>
<body>
<h1>Este es un título de nivel 1 y con el
elemento 'em' la palabra:
<em>Hola</em></h1>
<p>Todo este párrafo debe ser de color
azul ya que lo hereda del
elemento body.</p>
</body>
</html>
```

Este es un título de nivel 1 y con el elemento 'em' la palabra: *Hola*

Todo este párrafo debe ser de color azul ya que lo hereda del elemento body.

En este ejemplo hemos definido la siguiente regla para el elemento body:

```
body {  
  color:#0000ff;  
  font-family:verdana;  
}
```

Inicializamos la propiedad color con el valor de azul y la fuente de tipo verdana. Con esto todos los elementos contenidos en el body que no redefinan estas dos propiedades recibirán los valores aquí definidos. En este ejemplo la cabecera de primer nivel es decir h1 y el párrafo tienen como color el azul y la fuente es de tipo verdana.

Ahora bien en muchas situaciones podemos redefinir propiedades para elementos contenidos, veamos como podemos hacer que el texto contenido en los elementos em y p aparezcan de color distinto:

```
<!DOCTYPE html>
<html>
<head>
  <title>Problema</title>
  <meta charset="UTF-8">
<style>
body {
  color:#0000ff;
  font-family:verdana;
}
em {
  color:#008800;
}
p {
  color:#999999;
}
</style>
</head>
<body>
<h1>Este es un título de nivel 1 y con el elemento 'em' la palabra:
<em>Hola</em></h1>
<p>Todo este párrafo debe ser de color gris ya que lo redefine la
marca p y no lo hereda de la marca body.</p>
</body>
</html>
```

Este es un título de nivel 1 y con el elemento 'em' la palabra: *Hola*

Todo este párrafo debe ser de color gris ya que lo redefine la marca p y no lo hereda de la marca body.

Ahora hemos definido tres reglas, la primera igual que el problema anterior, define la propiedad color en azul y la fuente de tipo verdana para el elemento body:

```
body {  
    color:#0000ff;  
    font-family:verdana;  
}
```

La segunda regla define la propiedad color en verde para el elemento em, con esto no heredará el color azul del elemento body (que es el elemento padre):

```
em {  
    color:#008800;  
}
```

Algo similar hacemos con el elemento p para indicar que sea de color gris:

```
p {  
    color:#999999;  
}
```

Pero podemos ver que todos los elementos heredan la fuente verdana ya que ninguna lo sobrescribe.

Problema:

Confeccione una página que defina una regla para el elemento body e inicialice las propiedades color y font-family. Luego defina reglas de estilo para los elementos h1,h2 y h3 que redefinan la fuente con los valores: Times New Roman, Courier y Arial respectivamente. Imprima tres títulos, cada uno con los elementos h1,h2 y h3. Por último imprima un párrafo.

10 - Definición de un estilo en función del contexto.

Este otro recurso que provee las hojas de estilo en cascada (CSS) es la **definición de un estilo para un elemento HTML siempre y cuando el mismo esté contenido por otro elemento determinado**. Sólo en ese caso el estilo para dicho elemento se activará.

Supongamos que queremos que cuando disponemos un texto encerrado por el elemento strong dentro de un párrafo el color del mismo sea verde. Pero si el elemento strong está contenido por el elemento h1 el color debe ser rojo, luego la sintaxis del problema es:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>Problema</title>
```

```
  <meta charset="UTF-8">
```

```
<style>
```

```
p strong{
```

```
  color:#0000ff;
```

```
}
```

```
h1 strong{
```

```
  color:#ff0000;
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h1>Aquí tenemos un título de nivel uno, luego un bloque con  
el elemento strong debe aparecer <strong>así</strong></h1>
```

```
<p>
```

```
Luego si escribimos un párrafo y encerramos un bloque con el elemento strong  
debe aparecer <strong>así</strong>
```

```
</p>
```

```
</body>
```

```
</html>
```

Aquí tenemos un título de nivel uno, luego un bloque con el elemento strong debe aparecer así

Luego si escribimos un párrafo y encerramos un bloque con el elemento strong debe aparecer así

Es importante analizar la sintaxis para la definición de estilos en función del contexto.

Tenemos:

```
p strong{  
  color:#0000ff;  
}
```

Estamos diciendo **que todos los bloques de la página que contengan el elemento strong y que estén contenidas por el elemento p (párrafo) se pinten de azul**. Si bien hay dos bloques en la página que están encerrados por el elemento strong, solo uno se pinta de color azul, el otro bloque se pinta de color rojo, ya que tenemos:

```
h1 strong{  
  color:#ff0000;  
}
```

Es decir, activar el color rojo para el contenido encerrado por el elemento strong, siempre y cuando se encuentre contenido por el elemento h1.

No confundir con la sintaxis para definir reglas de estilo a dos elementos que se hace separando por coma los elementos HTML:

```
h1,strong{  
  color:#ff0000;  
}
```

Es decir que el texto contenido por los elementos h1 y strong deben aparecer de color rojo.

Se pueden definir estilos en función del contexto, con mayor precisión, como por ejemplo:

```
div h1 em {  
  color:#ff0000;  
}
```

Con esto estamos diciendo que **se debe pintar de color rojo el texto contenido por el elemento em siempre y cuando esté contenido en un elemento h1 y este a su vez dentro de un div.**

La página completa queda codificada de la siguiente forma:

```
<!DOCTYPE html>
<html>
<head>
  <title>Problema</title>
  <meta charset="UTF-8">
<style>
div h1 em {
  color:#ff0000;
}
</style>
</head>
```

```
<body>
<div>
<h1>Este es un titulo de nivel 1 dentro de
un <em>div</em></h1>
</div>
<h1>Este es un titulo de nivel 1 fuera de
un <em>div</em></h1>
</body>
</html>
```

Este es un titulo de nivel 1 dentro de un *div*

Este es un titulo de nivel 1 fuera de un *div*

Problema:

Definir distintas escalas de grises para los textos contenidos por el elemento "em" que se encuentren contenidos por los elementos h1,h2,h3,h4,h5 y h6.

La escala de grises que podemos utilizar es:

color:#dddddd; si está contenido en un h1

color:#cccccc; si está contenido en un h2

color:#bbbbbb; etc.

color:#aaaaaa;

color:#999999;

color:#888888;

Aparecen los bloques de más claro a más oscuro (cuando los tres valores son iguales (rojo,verde,azul) estamos en presencia de un gris (salvo el #000000 y el #ffffff))